دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

# داده‌ساختارها و الگوریتم‌ها

## جلسه چهاردهم: برخورد و حل آن با زنجیره‌سازی و آدرس‌دهی باز

دکتر شهرام گلزاری

پاییز ۱۴۰۵

# اهداف جلسه

در پایان این جلسه انتظار داریم که:

- با داشتن $n$، $m$ و طول زنجیره‌ها، ضریب بار و تعداد مورد انتظار عناصر بررسی‌شده در جستجوی موفق و ناموفق با زنجیره‌سازی را حساب کنیم.
- وضعیت جدول با وارسی خطی، وارسی درجه‌ی ۲ یا درهم‌سازی دوگانه را پس از دنباله‌ای از درج‌ها رسم و تعداد وارسی‌های هر درج و جست‌وجو را تعیین کنیم.
- تشخیص دهیم که یک دنباله‌ی وارسی (در وارسی درجه‌ی ۲ یا درهم‌سازی دوگانه) همه‌ی درایه‌های جدول را می‌پیماید یا نه.
- پس از دنباله‌ای از درج، حذف و جست‌وجو با نشانه‌ی deleted، وضعیت جدول و نتیجه‌ی هر جست‌وجو را تعیین کنیم و نشان دهیم چرا قرار دادن nil به جای آن خطا می‌دهد.
- احتمال پر شدن هر درایه‌ی خالی را در وارسی خطی حساب کنیم و میان روش‌های وارسی بر اساس تعداد دنباله‌ها و خوشه‌بندی انتخاب کنیم.

# یادآوری جلسه‌ی قبل

در جلسه‌ی قبل دیدیم که:

- هر الگوریتم جستجو با مقایسه در بدترین حالت $\Omega(\lg n)$ مقایسه لازم دارد.
- جدول آدرس‌دهی مستقیم هر سه عمل فرهنگ لغت را در بدترین حالت در زمان $O(1)$ انجام می‌دهد، اما فضای $\Theta(|U|)$ می‌گیرد.
- جدول درهم‌سازی با تابع $h$ فضا را به $\Theta(|K|)$ کاهش می‌دهد.

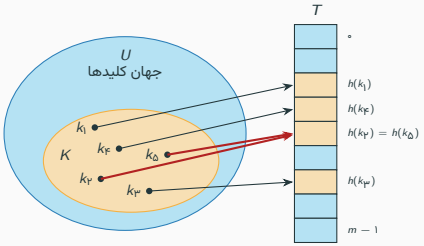
# برخورد

# برخورد

«یک مشکل وجود دارد، و آن اینکه دو کلید ممکن است به یک خانه هش شوند. این وضعیت را برخورد (collision) می‌نامیم. خوشبختانه روش‌های مؤثری برای حل تعارض ناشی از برخوردها وجود دارد.»

*CLRS, 4th ed.*، صفحه ۲۷۵ (ترجمه ماشینی)

در شکل، کلیدهای $k_2$ و $k_5$ به یک خانه نگاشته شده‌اند و برخورد دارند.




*CLRS, 4th ed.*، صفحه ۲۷۶ (بازترسیم شکل ۲.۱۱)

# اجتناب‌ناپذیری برخورد

«با این حال، چون $|U| > m$، دست‌کم دو کلید باید مقدار هش یکسانی داشته باشند و اجتناب کامل از برخوردها ناممکن است. بنابراین، هرچند یک تابع درهم‌ساز خوب‌طراحی‌شده و «تصادفی‌نما» می‌تواند تعداد برخوردها را کم کند، همچنان به روشی برای حل برخوردهایی که رخ می‌دهند نیاز داریم.»

*CLRS, 4th ed.*، صفحه‌های ۲۷۵ و ۲۷۶ (ترجمه ماشینی)

پاراگراف بالا کدام یک از اصول ریاضی را بیان می‌کند؟

# یافتن برخوردها

با $m = ۱۰$ و $h(k) = k \bmod ۱۰$، کلیدهای $۲۳, ۴۵, ۱۳, ۷۱, ۵۰, ۳۳$ را هش می‌کنیم:

| $k$ | ۲۳ | ۴۵ | ۱۳ | ۷۱ | ۵۰ | ۳۳ |
|---|---|---|---|---|---|---|
| $h(k)$ | ۳ | ۵ | ۳ | ۱ | ۰ | ۳ |

خانه اثر دارد و این یکی از دلایلی است که $k \bmod m$ تابع چندان خوبی نیست.

# یافتن برخوردها

با $m = ۱۰$ و $h(k) = k \bmod ۱۰$، کلیدهای $۲۳, ۴۵, ۱۳, ۷۱, ۵۰, ۳۳$ را هش می‌کنیم:

| $k$ | ۲۳ | ۴۵ | ۱۳ | ۷۱ | ۵۰ | ۳۳ |
|---|---|---|---|---|---|---|
| $h(k)$ | ۳ | ۵ | ۳ | ۱ | ۰ | ۳ |

کلیدهای ۲۳، ۱۳ و ۳۳ همگی به خانه‌ی ۳ درهم می‌شوند.

خانه اثر دارد و این یکی از دلایلی است که $k \bmod m$ تابع چندان خوبی نیست.

# یافتن برخوردها

با $m = ۱۰$ و $h(k) = k \bmod ۱۰$، کلیدهای $۲۳, ۴۵, ۱۳, ۷۱, ۵۰, ۳۳$ را هش می‌کنیم:

| $k$ | ۲۳ | ۴۵ | ۱۳ | ۷۱ | ۵۰ | ۳۳ |
|---|---|---|---|---|---|---|
| $h(k)$ | ۳ | ۵ | ۳ | ۱ | ۰ | ۳ |

کلیدهای ۲۳، ۱۳ و ۳۳ همگی به خانه‌ی ۳ درهم می‌شوند.

اکنون کلیدهای $۱۰, ۲۰, ۳۰, ۴۰$ را در نظر بگیرید: همه به خانه‌ی ۰ می‌روند. با $m = ۱۰$ فقط رقم آخر کلید در خانه اثر دارد و این یکی از دلایلی است که $k \bmod m$ تابع چندان خوبی نیست.

# درهم‌سازی یکنواخت مستقل

«یک تابع درهم‌ساز «ایده‌آل» $h$ برای هر ورودی ممکن $k$ در دامنه‌ی $U$، خروجی $h(k)$ دارد که عنصری است که به‌طور تصادفی، مستقل و یکنواخت از بازه‌ی $\{0, 1, \ldots, m-1\}$ انتخاب شده است. وقتی مقداری برای $h(k)$ به‌طور تصادفی انتخاب شد، هر فراخوانی بعدی $h$ با همان ورودی $k$ همان خروجی $h(k)$ را می‌دهد.»

*CLRS, 4th ed.*، صفحه ۲۷۶ (ترجمه ماشینی)

- چنین تابعی تابع درهم‌ساز یکنواخت مستقل (independent uniform hash function) یا پیشگوی تصادفی (random oracle) نامیده می‌شود.
- این یک فرض نظری برای تحلیل است و در عمل فقط تقریب زده می‌شود.

# حل برخورد با زنجیره‌سازی

# ایده‌ی زنجیره‌سازی

در زنجیره‌سازی (chaining)، «(...) هر خانه‌ی غیرخالی به یک لیست پیوندی اشاره می‌کند و همه‌ی عناصری که به یک خانه هش می‌شوند به لیست پیوندی آن خانه می‌روند. خانه‌ی $j$ اشاره‌گری به سر لیست همه‌ی عناصر ذخیره‌شده با مقدار هش $j$ دارد. اگر چنین عنصری نباشد، خانه‌ی $j$ مقدار nil دارد.»

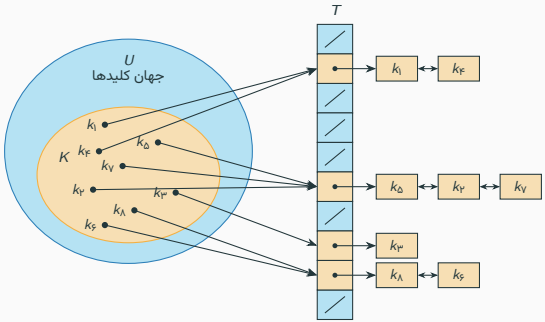
*CLRS, 4th ed.*، صفحه ۲۷۷ (ترجمه ماشینی)

## نگاه کلی

«در سطح بالا، می‌توان درهم‌سازی با زنجیره‌سازی را شکلی غیربازگشتی از تقسیم و حل دانست: مجموعه‌ی ورودی $n$ عنصری به‌طور تصادفی به $m$ زیرمجموعه، هر کدام با اندازه‌ی تقریبی $n/m$، تقسیم می‌شود.»

*CLRS, 4th ed.*، صفحه ۲۷۷ (ترجمه ماشینی)

# شکل زنجیره‌سازی




«حل برخورد با زنجیره‌سازی. هر خانه‌ی غیرخالی $T[j]$ جدول درهم‌سازی به یک لیست پیوندی از همه‌ی کلیدهایی اشاره می‌کند که مقدار هش آن‌ها $j$ است. برای مثال $h(k_1) = h(k_4)$ و $h(k_2) = h(k_5) = h(k_7)$.»

*CLRS, 4th ed.*، صفحه ۲۷۷ (بازترسیم شکل ۳.۱۱، ترجمه ماشینی شرح شکل)

# اعمال با زنجیره‌سازی

```
CHAINED-HASH-INSERT(T, x)
1  LIST-PREPEND(T[h(x.key)], x)

CHAINED-HASH-SEARCH(T, k)
1  return LIST-SEARCH(T[h(k)], k)

CHAINED-HASH-DELETE(T, x)
1  LIST-DELETE(T[h(x.key)], x)
```

*CLRS, 4th ed.*، صفحه ۲۷۸

- هر خانه یک لیست پیوندی است و هر عمل فقط روی لیست خانه‌ی $h(\cdot)$ انجام می‌شود.
- رویه‌های LIST-PREPEND، LIST-SEARCH و LIST-DELETE همان رویه‌های لیست پیوندی هستند که پیش‌تر دیدیم.

# زمان اجرای اعمال

«زمان اجرای درج در بدترین حالت $O(1)$ است. رویه‌ی درج تا حدی به این دلیل سریع است که فرض می‌کند عنصر $x$ که درج می‌شود از قبل در جدول نیست. (...) برای جستجو، زمان اجرا در بدترین حالت متناسب با طول لیست است. (...) اگر لیست‌ها دوطرفه باشند، حذف زمان $O(1)$ می‌گیرد.»

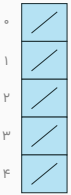
*CLRS, 4th ed.*، صفحه ۲۷۷ (ترجمه ماشینی)

**سوال**

چرا لیست دو طرفه؟

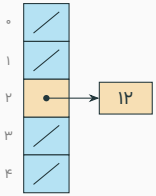
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

جدول خالی با ۵ خانه.



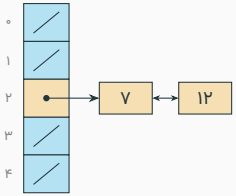
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۱۲: $h(۱۲) = ۲$.



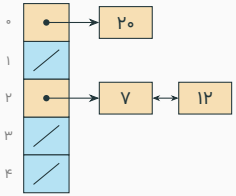
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۷: $h(۷) = ۲$، برخورد با ۱۲. عنصر ۷ به ابتدای لیست خانه‌ی ۲ افزوده می‌شود.



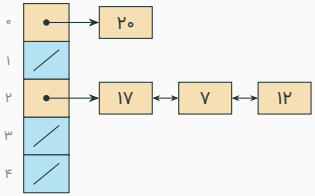
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۲۰: $h(۲۰) = ۰$.



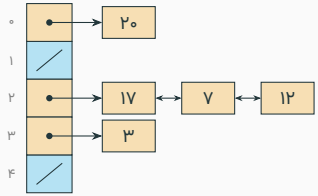
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۱۷: $h(۱۷) = ۲$. لیست خانه‌ی ۲ اکنون سه عنصر دارد.



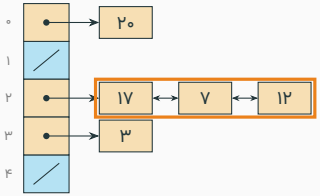
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

درج ۳: $h(۳) = ۳$.



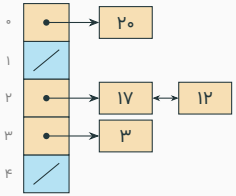
$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

جستجوی ۲۲: $h(۲۲) = ۲$. کلیدهای ۱۷، ۷ و ۱۲ مقایسه می‌شوند و پس از ۳ مقایسه، ناموفق بودن جستجو معلوم می‌شود.



$m = ۵$ و $h(k) = k \bmod ۵$. درج از ابتدای لیست انجام می‌شود.

حذف عنصر با کلید ۷ با داشتن اشاره‌گر به آن: پیوندهای ۱۷ و ۱۲ مستقیم به هم وصل می‌شوند، در زمان $O(۱)$.



# تحلیل زنجیره‌سازی

# ضریب بار

«با داشتن جدول درهم‌سازی $T$ با $m$ خانه که $n$ عنصر را ذخیره می‌کند، ضریب بار $\alpha$ (load factor) برای $T$ را $n/m$ تعریف می‌کنیم، یعنی میانگین تعداد عناصر ذخیره‌شده در یک زنجیره. تحلیل ما بر حسب $\alpha$ خواهد بود که می‌تواند کمتر از ۱، برابر ۱ یا بیشتر از ۱ باشد.»

*CLRS, 4th ed.*، صفحه ۲۷۸ (ترجمه ماشینی)

**مثال**

در ردیابی زنجیره‌سازی، پس از درج ۱۲، ۷، ۲۰، ۱۷ و ۳ در جدولی با $m = ۵$ خانه، $n = ۵$ و $\alpha = ۱$ است. اگر همین ۵ کلید در جدولی با $m = ۲$ خانه درج می‌شدند، $\alpha = ۲/۵$ بود.

# بدترین حالت زنجیره‌سازی

«رفتار درهم‌سازی با زنجیره‌سازی در بدترین حالت بسیار بد است: همه‌ی $n$ کلید به یک خانه هش می‌شوند و لیستی به طول $n$ می‌سازند. بنابراین زمان جستجو در بدترین حالت $\Theta(n)$ به‌علاوه‌ی زمان محاسبه‌ی تابع درهم‌ساز است، که بهتر از به‌کار بردن یک لیست پیوندی برای همه‌ی عناصر نیست. روشن است که جدول‌های درهم‌سازی را برای کارایی بدترین حالت آن‌ها به کار نمی‌بریم.»

*CLRS, 4th ed.*، صفحه ۲۷۸ (ترجمه ماشینی)

**مثال**

با $h(k) = k \bmod ۱۰$، کلیدهای $۱۰, ۲۰, \ldots, ۱۰n$ همگی به خانه‌ی ۰ می‌روند و جستجوی ناموفق کلید $۱۰(n+۱)$ هر $n$ عنصر را بررسی می‌کند.

# فرض یکنواختی و استقلال

«کارایی حالت میانگین درهم‌سازی به این بستگی دارد که تابع درهم‌ساز $h$ مجموعه‌ی کلیدهای ذخیره‌شونده را به‌طور میانگین چقدر خوب میان $m$ خانه پخش کند. (...) فعلاً فرض می‌کنیم که هر عنصر با احتمال برابر به هر یک از $m$ خانه هش شود، یعنی تابع درهم‌ساز یکنواخت $(\text{uniform})$ است. همچنین فرض می‌کنیم خانه‌ای که یک عنصر به آن هش می‌شود مستقل از خانه‌ی عناصر دیگر است. به بیان دیگر، فرض می‌کنیم از درهم‌سازی یکنواخت مستقل استفاده می‌کنیم.»

*CLRS, 4th ed.*، صفحه ۲۷۸ (ترجمه ماشینی)

«چون فرض شده است که هش کلیدهای متمایز مستقل از هم است، درهم‌سازی یکنواخت مستقل سراسری (universal) است: احتمال برخورد هر دو کلید متمایز $k_1$ و $k_2$ حداکثر $1/m$ است.»

*CLRS, 4th ed.*، صفحه ۲۷۸ (ترجمه ماشینی)

**مثال**

$n$ کلید $\binom{n}{2}$ جفت می‌سازند. با خطی بودن امید ریاضی، امید تعداد جفت‌های برخوردکننده حداکثر $m/\binom{n}{2}$ است. برای $n = m = 10$ این مقدار حداکثر $4.5 = 45/10$ است.

# طول زنجیره‌ها

«برای $j = ۰, ۱, \ldots, m - ۱$، طول لیست $T[j]$ را با $n_j$ نشان می‌دهیم، به‌طوری که

$$n = n_۰ + n_۱ + \cdots + n_{m-۱} \qquad (۱۱/۱)$$

و امید ریاضی $n_j$ برابر $\mathrm{E}[n_j] = \alpha = n/m$ است.»

*CLRS, 4th ed.*، صفحه ۲۷۸ و ۲۷۹ (ترجمه ماشینی)

**مثال**

در جدول ردیابی زنجیره‌سازی: $n_۰ = ۱$، $n_۱ = ۰$، $n_۲ = ۳$، $n_۳ = ۱$ و $n_۴ = ۰$. مجموع این طول‌ها $n = ۵$ است.

# هزینه‌ی جستجو

«فرض می‌کنیم زمان $O(1)$ برای محاسبه‌ی مقدار هش $h(k)$ کافی است، به‌طوری که زمان لازم برای جستجوی عنصری با کلید $k$ به‌طور خطی به طول $n_{h(k)}$ لیست $T[h(k)]$ بستگی دارد. (...) دو حالت را بررسی می‌کنیم. در حالت اول جستجو ناموفق است: هیچ عنصری در جدول کلید $k$ را ندارد. در حالت دوم جستجو با موفقیت عنصری با کلید $k$ را پیدا می‌کند.»

*CLRS, 4th ed.*، صفحه ۲۷۹ (ترجمه ماشینی)

**قضیه‌ی ۱.۱۱**

«در جدول درهم‌سازی‌ای که برخوردها در آن با زنجیره‌سازی حل می‌شوند، با فرض درهم‌سازی یکنواخت مستقل، جستجوی ناموفق به‌طور میانگین زمان $\Theta(1+\alpha)$ می‌گیرد.»

**اثبات.** «با فرض درهم‌سازی یکنواخت مستقل، هر کلید $k$ که از قبل در جدول نیست با احتمال برابر به هر یک از $m$ خانه هش می‌شود. زمان مورد انتظار جستجوی ناموفق کلید $k$ برابر زمان مورد انتظار جستجو تا انتهای لیست $T[h(k)]$ است که طول مورد انتظار آن $\mathrm{E}[n_{h(k)}] = \alpha$ است. بنابراین تعداد مورد انتظار عناصر بررسی‌شده در جستجوی ناموفق $\alpha$ است و زمان کل لازم (شامل زمان محاسبه‌ی $h(k)$) برابر $\Theta(1+\alpha)$ است.»

*CLRS, 4th ed.*، صفحه ۲۷۹ (ترجمه ماشینی)

# مثال جستجوی ناموفق

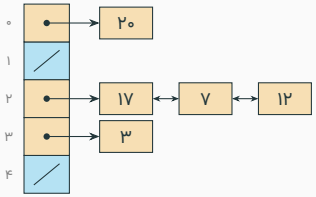
جدول ردیابی با $m = ۵$ و $n = ۵$، پس $\alpha = ۱$.

| $j$ | ۰ | ۱ | ۲ | ۳ | ۴ |
|---|---|---|---|---|---|
| $n_j$ | ۱ | ۰ | ۳ | ۱ | ۰ |

اگر کلید جستجوی ناموفق با احتمال برابر به هر خانه برود، میانگین تعداد عناصر بررسی‌شده

$$\frac{۱+۰+۳+۱+۰}{۵} = ۱ = \alpha$$

است.



# تفاوت جستجوی موفق

«جستجوی ناموفق با احتمال برابر به هر خانه‌ای از جدول درهم‌سازی می‌رود. اما جستجوی موفق نمی‌تواند به خانه‌ی خالی برود، زیرا برای عنصری است که در یکی از لیست‌های پیوندی وجود دارد. فرض می‌کنیم عنصر مورد جستجو با احتمال برابر هر یک از عناصر جدول باشد، پس هر چه لیست بلندتر باشد، احتمال اینکه جستجو برای یکی از عناصر آن باشد بیشتر است. با این همه، زمان مورد انتظار جستجو همچنان $\Theta(1+\alpha)$ به دست می‌آید.»

*CLRS, 4th ed.*، صفحه ۲۷۹ (ترجمه ماشینی)

# جستجوی موفق

## قضیه‌ی ۲.۱۱

«در جدول درهم‌سازی‌ای که برخوردها در آن با زنجیره‌سازی حل می‌شوند، با فرض درهم‌سازی یکنواخت مستقل، جستجوی موفق به‌طور میانگین زمان $\Theta(1+\alpha)$ می‌گیرد.»

**ایده‌ی اثبات.** «تعداد عناصری که در جستجوی موفق عنصر $x$ بررسی می‌شوند یکی بیشتر از تعداد عناصری است که در لیست $x$ پیش از $x$ آمده‌اند. چون عناصر جدید در ابتدای لیست قرار می‌گیرند، عناصر پیش از $x$ در لیست همگی پس از درج $x$ درج شده‌اند.»

*CLRS, 4th ed.*، صفحه ۲۷۹ و ۲۸۰ (ترجمه ماشینی)

## مثال

در لیست خانه‌ی ۲ (۱۷، ۷، ۱۲)، کلید ۱۲ اول درج شده است. جستجوی ۱۲ سه عنصر را بررسی می‌کند: دو عنصر ۱۷ و ۷ که پس از آن درج شده‌اند و خود ۱۲.

# متغیرهای تصادفی شاخص

«فرض کنید $x_i$ نشان‌دهنده‌ی $i$اُمین عنصر درج‌شده در جدول باشد، برای $i = ۱, ۲, \ldots, n$، و $k_i = x_i.key$.»
برای هر خانه‌ی $q$ و هر دو کلید متمایز $k_i$ و $k_j$:

$$X_{ijq} = \mathrm{I}\{\text{ the search is for} x_i,\ h(k_i) = q,\ h(k_j) = q\}$$

«یعنی $X_{ijq} = ۱$ وقتی کلیدهای $k_i$ و $k_j$ در خانه‌ی $q$ برخورد کنند و جستجو برای عنصر $x_i$ باشد.»

*CLRS, 4th ed.*، صفحه ۲۸۰ (ترجمه ماشینی)

احتمال جستجوی $x_i$ برابر $۱/n$ و احتمال هر یک از $h(k_i) = q$ و $h(k_j) = q$ برابر $۱/m$ است و این پیشامدها مستقل‌اند.
پس

$$\Pr\{X_{ijq} = ۱\} = \frac{۱}{nm^۲}, \qquad \mathrm{E}[X_{ijq}] = \frac{۱}{nm^۲}.$$

# شمارش عناصر پیشین

$$Y_j = \mathrm{I}\{x_j \text{ appears in a list prior to the element being searched for}\} = \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq},$$

«زیرا حداکثر یکی از $X_{ijq}$ها برابر ۱ است، یعنی وقتی که عنصر مورد جستجوی $x_i$ به همان لیست $x_j$ (لیستی که خانه‌ی $q$ به آن اشاره می‌کند) تعلق داشته باشد و $i < j$ (پس $x_i$ در لیست پس از $x_j$ می‌آید).»

*CLRS, 4th ed.*، صفحه ۲۸۰ (ترجمه ماشینی)

$Z$ تعداد عناصری است که در لیست پیش از عنصر مورد جستجو آمده‌اند:

$$Z = \sum_{j=1}^{n} Y_j.$$

چون خود عنصر مورد جستجو هم بررسی می‌شود، باید $\mathrm{E}[Z+1]$ را حساب کنیم.

$$\begin{aligned}
\mathrm{E}[Z+1] &= \mathrm{E}\Big[1+\sum_{j=1}^{n} Y_j\Big] = 1+\mathrm{E}\Big[\sum_{j=1}^{n}\sum_{q=0}^{m-1}\sum_{i=1}^{j-1} X_{ijq}\Big] \\
&= 1+\mathrm{E}\Big[\sum_{q=0}^{m-1}\sum_{j=1}^{n}\sum_{i=1}^{j-1} X_{ijq}\Big] = 1+\sum_{q=0}^{m-1}\sum_{j=1}^{n}\sum_{i=1}^{j-1}\mathrm{E}[X_{ijq}] \\
&= 1+\sum_{q=0}^{m-1}\sum_{j=1}^{n}\sum_{i=1}^{j-1}\frac{1}{nm^2} = 1+m\cdot\frac{n(n-1)}{2}\cdot\frac{1}{nm^2} \\
&= 1+\frac{n-1}{2m} = 1+\frac{\alpha}{2}-\frac{\alpha}{2n}.
\end{aligned}$$

گام چهارم از خطی بودن امید ریاضی و گام ششم از $\sum_{j=1}^{n}(j-1) = n(n-1)/2$ به دست می‌آید.

*CLRS, 4th ed.*، صفحه‌های ۲۸۰ و ۲۸۱

# مثال جستجوی موفق

«بنابراین زمان کل لازم برای جستجوی موفق (شامل زمان محاسبه‌ی تابع درهم‌ساز) $\Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha)$ است.»

*CLRS, 4th ed.*، صفحه ۲۸۱ (ترجمه ماشینی)

در جدول ردیابی ($n = m = 5$) فرمول قضیه‌ی ۲.۱۱ می‌دهد:

$$1 + \frac{\alpha}{2} - \frac{\alpha}{2n} = 1 + \frac{1}{2} - \frac{1}{10} = 1/4.$$

| کلید | ۲۰ | ۱۷ | ۷ | ۱۲ | ۳ |
|---|---|---|---|---|---|
| عناصر بررسی‌شده | ۱ | ۱ | ۲ | ۳ | ۱ |

میانگین واقعی در این جدول $8/5 = 1/6$ است. عدد ۱٫۴ امید ریاضی روی انتخاب تصادفی تابع درهم‌ساز است و هر جدول مشخص می‌تواند از آن کمتر یا بیشتر باشد.

# معنای تحلیل

«اگر تعداد عناصر جدول حداکثر متناسب با تعداد خانه‌های جدول درهم‌سازی باشد، $n = O(m)$ و در نتیجه $\alpha = n/m = O(m)/m = O(1)$. پس جستجو به‌طور میانگین زمان ثابت می‌گیرد. (...) می‌توانیم همه‌ی اعمال فرهنگ لغت را به‌طور میانگین در زمان $O(1)$ پشتیبانی کنیم.»

*CLRS, 4th ed.*، صفحه ۲۸۱ (ترجمه ماشینی)

# آدرس‌دهی باز

# آدرس‌دهی باز

«در «آدرس‌دهی باز» (open hashing)، یک آرایه به‌اندازه‌ی $m$ به‌عنوان جدول درهم‌سازی استفاده می‌شود که حداکثر $m$ عنصر در آن جای می‌گیرد و در هر درایه، فقط یک عنصر ذخیره می‌شود. بنابراین، برای استفاده از این روش، تعداد بیشینه‌ی عناصر را باید از قبل بدانیم، یا آن‌که از «جدول درهم‌سازی پویا» (dynamic hash table) استفاده کنیم که در آن اندازه‌ی جدول برحسب نیاز، کم یا زیاد (معمولاً نصف یا ۲ برابر) می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

برخلاف زنجیره‌سازی، در آدرس‌دهی باز هیچ عنصری بیرون از جدول نگهداری نمی‌شود، پس $n \leq m$ و $\alpha \leq ۱$.

«برای درج یک عنصر در آدرس‌دهی باز، باید تعدادی از درایه‌های جدول «وارسی» (probing) شوند تا جای مناسب برای درج یک عنصر پیدا شود. برای این‌کار تابع درهم‌سازی را به این صورت تعریف می‌کنیم:

$$h : U \times \{۰, ۱, \ldots, m-۱\} \to \{۰, ۱, \ldots, m-۱\}.$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

# دنباله‌ی وارسی

«در آدرس‌دهی باز و برای یک کلید $k$ به درایه‌های زیر که به‌صورت متوالی وارسی می‌شوند، «دنباله‌ی وارسی» $k$ (probe sequence) می‌گوییم.

$$\langle h(k, ۰), h(k, ۱), \ldots, h(k, m-۱) \rangle$$

برای جست‌وجوی یک کلید، دنباله‌ی وارسی آن دنبال می‌شود تا آن کلید یافت شود، وگرنه کلید در جدول موجود نیست.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

# درج در آدرس‌دهی باز

«برای درج، دنباله‌ی وارسی بررسی می‌شود تا اولین درایه‌ی خالی پیدا شود و عنصر مورد نظر در آن‌جا درج می‌شود. اگر چنین جای خالی‌ای پیدا نشد، در آن‌صورت پیام خطای overflow چاپ می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

«توجه کنید که تابع درهم‌سازی $h(k, i)$ ممکن است طوری طراحی شده باشد که دنباله‌ی وارسی یک کلید، به یک درایه بیش از یک بار برود و در نتیجه، به بعضی از درایه‌ها اصلاً رجوع نکند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

**مثال**

با $m = ۴$، اگر دنباله‌ی وارسی کلید $k$ برابر $\langle ۲, ۰, ۲, ۰ \rangle$ باشد و درایه‌های ۰ و ۲ پر باشند، درج $k$ با خطای سرریز پایان می‌یابد، هرچند درایه‌های ۱ و ۳ خالی‌اند.

# درج و جست‌وجو

```
HASH-SEARCH(T, k)
1  i = 0
2  repeat j = h(k, i)
3      if T[j] == k
4          return j
5      i = i + 1
6  until i == m
7  return NIL
```

```
HASH-INSERT(T, k)
1  i = 0
2  repeat j = h(k, i)
3      if T[j] == NIL
4          T[j] = k
5          return j
6      else i = i + 1
7  until i == m
8  error "hash table overflow"
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹، با نمادگذاری *CLRS*

HASH-SEARCH در بدترین حالت همه‌ی $m$ درایه‌ی دنباله‌ی وارسی را بررسی می‌کند، حتی اگر به درایه‌ی خالی برسد.

# توقف زودتر جست‌وجو

«دقت کنید که اگر به‌جای $i = m$ دستور $\text{until } i = m$ دستور $\text{until } A[j] = \text{null or } i = m$ را قرار دهیم، ظاهراً موجب تسریع عمل درج می‌شود (...) اگر عنصر در جدول نباشد لزومی به گشتن همه‌ی دنباله‌ی وارسی نیست و اولین درایه‌ی خالی جواب منفی را تولید می‌کند. اما در صورت این تغییر، اگر جست‌وجو با حذف همراه شود، چنان‌چه خواهیم دید، ممکن است مشکل ایجاد کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۸

این تغییر به رویه‌ی HASH-SEARCH مربوط است: با آن، جست‌وجوی ناموفق با رسیدن به نخستین درایه‌ی خالی پایان می‌یابد.

# حذف ساده

«برای حذف یک عنصر، ابتدا آن‌را پیدا می‌کنیم و مقدار آن درایه را برابر تهی قرار می‌دهیم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹

```
    HASH-DELETE(T, k)
1   i = HASH-SEARCH(T, k)
2   if i ≠ NIL
3       T[i] = NIL
4   else error "key k not found"
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹، با نمادگذاری *CLRS*

# نشانه‌ی حذف

«اما با انجام تغییرات زیر می‌توانیم کار جست‌وجو را کمی سریع‌تر کنیم. پس از هر عمل حذف، به‌جای این‌که در درایه عنصر حذف‌شده، مقدار null قرار دهیم، مقدار خاص “deleted” را می‌نویسیم. در این صورت، درایه‌ی “deleted” برای جست‌وجو مانند درایه‌ی پر است. اما برای درج، اگر عنصر قبلاً وجود نداشته باشد، این درایه مانند درایه‌ی تهی محسوب می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۷۹

```
    HASH-DELETE-NEW(T, k)
1   i = HASH-SEARCH(T, k)
2   if i ≠ NIL
3       T[i] = DELETED
4   else error “key k not found”
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۰، با نمادگذاری *CLRS*

# جست‌وجو و درج تازه

```
HASH-INSERT-NEW(T, k)
1  if HASH-SEARCH(T, k) ≠ NIL
2     error "item already exists"
3  i = 0
4  repeat j = h(k, i)
5     if T[j] == NIL or T[j] == DELETED
6        T[j] = k
7        return j
8     else i = i + 1
9  until i == m
10 error "hash table overflow"
```

```
HASH-SEARCH-NEW(T, k)
1  i = 0
2  repeat j = h(k, i)
3     if T[j] == k
4        return j
5     i = i + 1
6  until T[j] == NIL or i == m
7  return NIL
```

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۰، با نمادگذاری *CLRS*

HASH-SEARCH-NEW در درایه‌ی deleted متوقف نمی‌شود، زیرا deleted با nil برابر نیست.

# روش‌های وارسی

# سه روش وارسی

«سه روش برای محاسبه‌ی دنباله‌ی وارسی به‌کار می‌رود. این روش‌ها تضمین می‌کنند که برای هر کلید $k$ دنباله‌ی

$$\langle h(k,0), h(k,1), \ldots, h(k,m-1)\rangle$$

یک جای‌گشت از $\langle 0,1,\ldots,m-1\rangle$ باشد. البته هیچ‌یک از این روش‌ها، فرض درهم‌سازی یکنوا را ارضا نمی‌کند، چون هیچ‌کدام قادر به تولید بیش از $m^2$ دنباله‌ی وارسی نیست، که بسیار کم‌تر از $m!$ است. درهم‌سازی دوگانه بیش‌ترین تعداد دنباله‌ها را تولید می‌کند و گاهی از دو روش دیگر بهتر است.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۱

در درهم‌سازی یکنوا، دنباله‌ی وارسی هر کلید با احتمال برابر یکی از $m!$ جایگشت $\langle 0,1,\ldots,m-1\rangle$ است.

«با فرض داشتن یک تابع درهم‌سازی عادی $h' : U \to \{0, 1, \ldots, m-1\}$ که از آن به نام تابع درهم‌سازی کمکی (auxiliary) یاد می‌کنیم، روش «وارسی خطی» (linear probing) از تابع زیر برای $i = 0, 1, \ldots, m-1$ استفاده می‌کند:

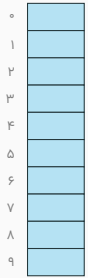
$$h(k, i) = (h'(k) + i) \bmod m \qquad (۴-۵)$$

به‌ازای یک کلید $k$ ابتدا $T[h'(k)]$ را وارسی می‌کنیم، سپس به $T[h'(k)+1]$ و بعد به $T[h'(k)+2]$ می‌رویم و این کار را به‌صورت حلقوی ادامه می‌دهیم تا به $T[h'(k)-1]$ برسیم. روشن است که این روش فقط $m$ دنباله‌ی وارسی مجزا تولید می‌کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

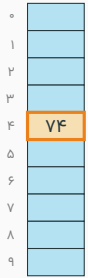
جدول خالی با ۱۰ درایه.




کلیدها و ترتیب درج از $CLRS,\ 4th\ ed.$، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

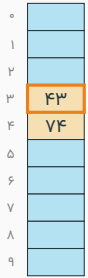
درج ۷۴: $h'(۷۴) = ۴$. یک وارسی.




کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

درج ۴۳: $h'(۴۳) = ۳$. یک وارسی.




کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

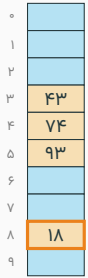
درج ۹۳: $h'(۹۳) = ۳$. درایه‌های ۳ و ۴ پرند و ۹۳ پس از ۳ وارسی در درایه‌ی ۵ قرار می‌گیرد.




کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

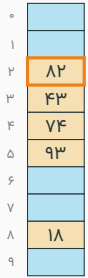
درج ۱۸: $h'(۱۸) = ۸$. یک وارسی.




کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

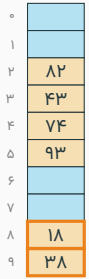
درج ۸۲: $h'(۸۲) = ۲$. یک وارسی.




کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

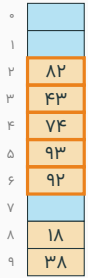
درج ۳۸: $h'(۳۸) = ۸$. درایه‌ی ۸ پر است و ۳۸ پس از ۲ وارسی در درایه‌ی ۹ قرار می‌گیرد.




کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

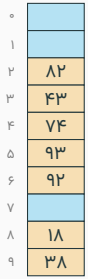
$m = ۱۰$ و $h'(k) = k \bmod ۱۰$. ترتیب درج: $۹۲, ۳۸, ۸۲, ۱۸, ۹۳, ۴۳, ۷۴$.

درج ۹۲: $h'(۹۲) = ۲$. درایه‌های ۲ تا ۵ پرند و ۹۲ پس از ۵ وارسی در درایه‌ی ۶ قرار می‌گیرد.




کلیدها و ترتیب درج از *CLRS, 4th ed.*، صفحه ۳۰۳، شکل ۶.۱۱

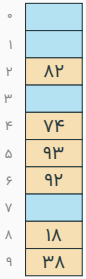
جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:




کلید ۹۳ هنگام درج، درایه‌های ۳ و ۴ را پر یافته و در درایه‌ی ۵ قرار گرفته است.

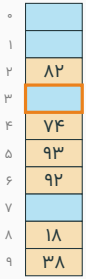
جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:

$\textsc{Hash-Delete}(T, 43)$: درایه‌ی ۳ برابر nil می‌شود.



# ردیابی حذف و جست‌وجو

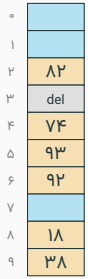
جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:




HASH-SEARCH-NEW$(T, 93)$: $h(۹۳, ۰) = ۳$ و $T[۳] = \text{nil}$، پس جست‌وجو با پاسخ nil پایان می‌یابد، هرچند ۹۳ در درایه‌ی ۵ است.

# ردیابی حذف و جست‌وجو

جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:




به جای آن، HASH-DELETE-NEW$(T, 43)$ در درایه‌ی ۳ مقدار deleted می‌نویسد.

# ردیابی حذف و جست‌وجو

جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:




HASH-SEARCH-NEW$(T, 93)$: درایه‌ی ۳ برابر deleted است و جست‌وجو ادامه می‌یابد. ۹۳ پس از ۳ وارسی در درایه‌ی ۵ پیدا می‌شود.

جدول اسلاید قبل، با وارسی خطی و $h'(k) = k \bmod ۱۰$:

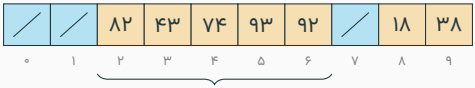



HASH-INSERT-NEW$(T, 13)$: HASH-SEARCH هر ۱۰ درایه را وارسی می‌کند و ۱۳ را نمی‌یابد. سپس از $h(۱۳, ۰) = ۳$ شروع می‌کنیم و چون $T[۳] = \text{deleted}$، کلید ۱۳ همان‌جا درج می‌شود.

# خوشه‌بندی اولیه

«پیاده‌سازی وارسی خطی ساده است. ولی این روش مشکلی به‌نام «خوشه‌بندی اولیه» (primary clustering) دارد که موجب می‌شود تا در جدول، بخش‌های پر، دنبال هم ظاهر شوند که هر کدام را یک خوشه می‌گوییم. یک درایه‌ی خالی که بلافاصله بعد از $i$ درایه‌ی پر دنبال هم قرار گیرد، در مرحله‌ی بعد و با احتمال $(i+1)/m$ پر می‌شود و به این ترتیب، به مرور زمان خوشه‌ها بزرگ‌تر شده و زمان جست‌وجو هم طولانی‌تر می‌شود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۱




در جدول ردیابی، درایه‌های خالی ۰، ۱ و ۷ هستند. درایه‌ی ۷ پس از ۵ درایه‌ی پر با احتمال ۶/۱۰، درایه‌ی ۰ پس از ۲ درایه‌ی پر (۸ و ۹) با احتمال ۳/۱۰ و درایه‌ی ۱ با احتمال ۱/۱۰ پر می‌شود.

# وارسی درجه‌ی ۲

««وارسی درجه‌ی ۲» (quadratic probing) از تابع درهم‌سازی زیر برای ($i = ۰, ۱, \ldots, m - ۱$) استفاده می‌کند:

$$h(k, i) = (h'(k) + c_۱ i + c_۲ i^۲) \bmod m \qquad (۵\text{-}۵)$$

که در آن $h'$ تابع درهم‌سازی کمکی و $c_۱, c_۲$ ($c_۲ \neq ۰$) ثابت‌های کمکی هستند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۲

«این روش بسیار بهتر از وارسی خطی عمل می‌کند و مشکل خوشه‌بندی را ندارد. البته برای این‌که همه‌ی $m$ درایه‌ی جدول مورد استفاده قرار گیرند، مقادیر $c_1$، $c_2$ و $m$ باید به‌صورت خاص انتخاب شوند. همچنین، اگر وارسی اولیه‌ی دو کلید مشابه باشند، دنباله‌ی وارسی‌های آن‌ها هم یک‌سان است

$$h(k_1, 0) = h(k_2, 0) \Longrightarrow h(k_1, i) = h(k_2, i).$$

»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۲

# مثال وارسی درجه‌ی ۲

دنباله‌ی وارسی کلیدی با $h'(k) = ۰$:

| $m = ۱۰$، $c_۱ = ۰$، $c_۲ = ۱$ | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $i$ | ۰ | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ | ۷ | ۸ | ۹ |
| $h(k, i)$ | ۰ | ۱ | ۴ | ۹ | ۶ | ۵ | ۶ | ۹ | ۴ | ۱ |

| $m = ۸$، $c_۱ = c_۲ = ۱/۲$ | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| $i$ | ۰ | ۱ | ۲ | ۳ | ۴ | ۵ | ۶ | ۷ |
| $h(k, i)$ | ۰ | ۱ | ۳ | ۶ | ۲ | ۷ | ۵ | ۴ |

در انتخاب اول فقط ۶ درایه از ۱۰ درایه وارسی می‌شود. در انتخاب دوم $h(k, i) = i(i+۱)/۲ \bmod ۸$ است و هر ۸ درایه وارسی می‌شود.

# درهم‌سازی دوگانه

«درهم‌سازی دوگانه (double hashing) یک روش وارسی است که در آن تابع درهم‌سازی به‌صورت زیر است:

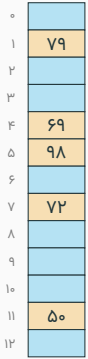
$$h(k, i) = (h_1(k) + i\, h_2(k)) \bmod m$$

که $h_1$ و $h_2$ تابع‌های درهم‌سازی کمکی هستند. اولین وارسی در $T[h_1(k)]$ انجام می‌شود و وارسی‌های بعدی به درایه‌هایی با فاصله‌ی $h_2(k)$ (البته به هنگ $m$) می‌رود.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۲

# درج با درهم‌سازی دوگانه

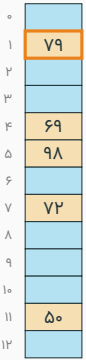
«در این مثال، $m = ۱۳$ و $h_۱(k) = k \bmod ۱۳$ و $h_۲(k) = ۱ + (k \bmod (۱۱$» کلید ۱۴ را درج می‌کنیم.




داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

# درج با درهم‌سازی دوگانه

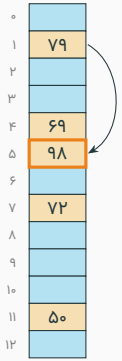
وارسی اول: $h_1(۱۴) = ۱$. درایه‌ی ۱ پر است.




داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

# درج با درهم‌سازی دوگانه

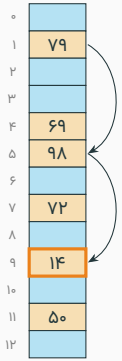
$h_۲(۱۴) = ۱ + ۳ = ۴$. وارسی دوم: $(۱ + ۴) \bmod ۱۳ = ۵$. درایه‌ی ۵ هم پر است.




داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

# درج با درهم‌سازی دوگانه

«چون $۱۴ \equiv ۱ \pmod{۱۳}$ و $۱۴ \equiv ۳ \pmod{۱۱}$، بنابراین کلید ۱۴، پس از آن‌که درایه‌های ۱ و ۵ وارسی شد و پر بودند، در درایه‌ی خالی ۹ درج می‌شود.»




داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳ (بازترسیم شکل ۵-۵)

«در طراحی این درهم‌سازی، باید مقدار $h_۲(k)$ نسبت به اندازه‌ی جدول، یا $m$ اول باشد تا این‌که کلیه‌ی درایه‌های جدول وارسی شوند. یک روش مناسب برای اعمال این خاصیت، انتخاب $m$ به‌صورت توان ۲ و طراحی $h_۲$ به‌طوری است که همیشه یک عدد فرد تولید کند. یک روش دیگر آن است که $m$ عدد اول باشد و $h_۲$ همیشه یک عدد صحیح و مثبت کم‌تر از $m$ را تولید کند.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه‌های ۲۸۲ و ۲۸۳

**مثال**

با $m = ۱۰$، $h_۱(k) = ۳$ و $h_۲(k) = ۵$ دنباله‌ی وارسی $۳, ۸, ۳, ۸, \ldots$ است و فقط ۲ درایه وارسی می‌شود. در حالت کلی تعداد درایه‌های وارسی‌شده $m/\gcd(h_۲(k), m)$ است.

# یک انتخاب رایج

«مثلاً، $m$ می‌تواند یک عدد اول و $h_۱$ و $h_۲$ به‌صورت زیر باشند:

$$h_۱(k) = k \bmod m, \qquad h_۲(k) = ۱ + (k \bmod m'),$$

که در آن $m'$ عددی کوچک‌تر و نزدیک به $m$ (مثلاً $m - ۱$) است. مثلاً اگر $k = ۱۲۳۴۵۶$ و $m = ۷۰۱$ و $m' = ۷۰۰$ باشد، داریم $h_۱(k) = ۸۰$ و $h_۲(k) = ۲۵۷$. بنابراین ابتدا درایه‌ی ۸۰ و سپس درایه‌های با فاصله‌ی ۲۵۷ از آن (به هنگ $m$) را وارسی می‌کنیم.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۳

چند وارسی بعدی: $h(k, ۱) = ۳۳۷$، $h(k, ۲) = ۵۹۴$ و $h(k, ۳) = (۸۰ + ۷۷۱) \bmod ۷۰۱ = ۱۵۰$.

# مقایسه‌ی روش‌های وارسی

«درهم‌سازی دوگانه نسبت به وارسی خطی و وارسی درجه‌ی ۲ بهتر عمل می‌کند، چون به تعداد $\Theta(m^۲)$ (در مقایسه با $\Theta(m)$) دنباله‌ی وارسی تولید می‌کند. بنابراین به‌نظر می‌رسد کارایی این درهم‌سازی به کارایی آرمانی درهم‌سازی یکنوا نزدیک باشد.»

داده‌ساختارها و مبانی الگوریتم‌ها، محمد قدسی، صفحه ۲۸۳

# منابع

# منابع


- محمد قدسی، داده‌ساختارها و مبانی الگوریتم‌ها، انتشارات فاطمی، چاپ ۱۳۹۷
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.